Bài tập: Design Patterns

Singleton:

Trong hệ thống ô tô, ECU chịu trách nhiệm điều khiển động cơ, bao gồm việc kiểm soát tốc độ động cơ, nhiệt độ và các thông số quan trọng khác. Vì chỉ có duy nhất một ECU trong xe, nó cần được triển khai như một Singleton để đảm bảo có thể truy cập nó từ bất kỳ phần nào của hệ thống mà không cần khởi tạo nhiều lần.

Yêu cầu:

1. Thiết kế lớp EngineControlUnit theo mẫu Singleton:

- **Mục đích**: Lớp EngineControlUnit mô phỏng hệ thống điều khiển động cơ của xe (ECU). Mẫu thiết kế Singleton đảm bảo rằng trong toàn bộ chương trình chỉ tồn tại duy nhất một instance của ECU. Điều này là cần thiết trong các hệ thống automotive, nơi chỉ có một ECU điều khiển toàn bộ động cơ, và bất kỳ bộ phận nào cũng có thể truy cập được nó.
- **Cấu trúc Singleton**:
    - **Phương thức khởi tạo (constructor)**: Được đặt là private để ngăn chặn việc khởi tạo instance mới từ bên ngoài.
    - **Biến static instance**: Dùng để lưu trữ instance duy nhất của lớp. Biến này được khởi tạo là nullptr hoặc giá trị ban đầu trong phần khai báo.
    - **Phương thức getInstance()**: Phương thức static kiểm tra xem instance đã tồn tại chưa, nếu chưa thì tạo mới instance, nếu đã tồn tại thì trả về instance hiện tại.

2. Các chức năng cần thiết trong lớp EngineControlUnit:

- **setEngineSpeed(int speed)**:
    - Chức năng này dùng để thiết lập tốc độ động cơ (RPM). Giá trị tốc độ phải nằm trong khoảng từ **0 đến 6000 RPM** để đảm bảo an toàn và tính thực tế trong mô phỏng.
    - Nếu tốc độ không hợp lệ (nằm ngoài khoảng cho phép), hàm sẽ in ra thông báo cảnh báo và không thay đổi giá trị hiện tại.
- **getEngineSpeed()**:

    - Trả về tốc độ động cơ hiện tại. Điều này giúp các bộ phận khác trong hệ thống có thể truy cập và lấy thông tin về tốc độ động cơ bất cứ lúc nào.
- **setEngineTemperature(float temperature)**:
    - Chức năng này dùng để thiết lập nhiệt độ động cơ (Celsius). Giá trị nhiệt độ phải nằm trong khoảng từ **-20 đến 120 độ C** để mô phỏng phạm vi nhiệt độ thực tế của động cơ.
    - Nếu nhiệt độ không hợp lệ, hàm sẽ đưa ra cảnh báo và không cập nhật giá trị.
- **getEngineTemperature()**:
    - Trả về nhiệt độ động cơ hiện tại. Các hệ thống giám sát hoặc bộ phận điều khiển khác có thể cần thông tin này để kiểm tra tình trạng động cơ.
- **diagnostics()**:
    - Chức năng này sẽ chạy các chẩn đoán cơ bản trên động cơ, in ra các thông số hiện tại như tốc độ và nhiệt độ động cơ.
    - **Kiểm tra quá nhiệt**: Nếu nhiệt độ động cơ vượt quá **100 độ C**, chương trình sẽ cảnh báo rằng động cơ đang quá nhiệt. Đây là một kiểm tra quan trọng trong automotive để đảm bảo rằng động cơ không bị hư hại do nhiệt độ quá cao.

3. Chương trình chính (main) mô phỏng việc điều khiển ECU:

Trong chương trình chính:

- **Khởi tạo và sử dụng Singleton**: Sử dụng phương thức getInstance() để lấy instance của lớp EngineControlUnit. Điều này đảm bảo rằng tất cả các lần truy cập đến ECU đều dùng chung một đối tượng duy nhất.
- **Thiết lập và lấy giá trị tốc độ, nhiệt độ**: Gọi các phương thức setEngineSpeed() và setEngineTemperature() để mô phỏng việc thay đổi giá trị tốc độ và nhiệt độ của động cơ. Các giá trị này sẽ được giới hạn để đảm bảo rằng không có tốc độ hoặc nhiệt độ không hợp lệ.
- **Chạy chẩn đoán**: Gọi hàm diagnostics() để thực hiện kiểm tra và in ra các thông số hiện tại của động cơ, bao gồm cả việc kiểm tra động cơ có đang quá nhiệt hay không.

4. Nâng cao bài tập:

- **Kiểm tra và cảnh báo quá nhiệt**:

    - Chức năng isOverheating() được thêm vào để kiểm tra nhiệt độ động cơ có vượt ngưỡng **100 độ C** không. Nếu vượt ngưỡng, chương trình sẽ in ra cảnh báo về tình trạng quá nhiệt của động cơ, mô phỏng một cơ chế cảnh báo quan trọng trong các hệ thống điều khiển xe.
- **Lưu trữ lịch sử dữ liệu**:
    - Thêm khả năng lưu trữ lịch sử của tốc độ và nhiệt độ động cơ. Mỗi khi giá trị mới được thiết lập, nó sẽ được ghi lại vào lịch sử. Khi chạy chẩn đoán, chương trình sẽ in ra danh sách các giá trị lịch sử để người dùng hoặc kỹ sư có thể theo dõi sự thay đổi của động cơ theo thời gian.
- **Giao diện với các hệ thống khác**:
    - Trong thực tế, ECU thường giao tiếp với các hệ thống khác trong xe, như hệ thống phanh hoặc hệ thống kiểm soát nhiên liệu. Có thể thêm các phương thức để mô phỏng sự tương tác này, chẳng hạn như:
        - **setFuelInjectionRate()**: Điều chỉnh mức nhiên liệu được bơm vào động cơ dựa trên tốc độ và tình trạng của động cơ.
        - **brakeControlInteraction()**: Giả lập giao tiếp với hệ thống phanh, giảm tốc độ động cơ khi hệ thống phanh kích hoạt.

Observer:

Trong hệ thống ô tô, nhiều thành phần có thể cần được thông báo khi một sự kiện hoặc trạng thái quan trọng thay đổi. Ví dụ, khi nhiệt độ động cơ vượt quá một mức nhất định, các hệ thống khác như hệ thống làm mát, cảnh báo trên bảng điều khiển, và ECU (Engine Control Unit) cần được thông báo để thực hiện các hành động thích hợp. Mẫu thiết kế **Observer** cho phép một đối tượng (Subject) thông báo cho nhiều đối tượng khác (Observer) khi trạng thái của nó thay đổi.

Yêu cầu:

1. **Thiết kế một lớp Subject đại diện cho động cơ (Engine)**, nơi có thể theo dõi và thông báo cho các đối tượng quan sát (Observers) khi có thay đổi về trạng thái, chẳng hạn như nhiệt độ hoặc tốc độ động cơ.
2. **Thiết kế các lớp Observer** đại diện cho các hệ thống liên quan, bao gồm:

- **Hệ thống làm mát (CoolingSystem)**: Hệ thống này cần được thông báo khi nhiệt độ động cơ vượt quá mức an toàn để kích hoạt quạt làm mát.
- **Hệ thống cảnh báo (WarningSystem)**: Cảnh báo trên bảng điều khiển sẽ được kích hoạt khi có bất kỳ sự cố nào liên quan đến nhiệt độ hoặc tốc độ động cơ.
- **ECU (EngineControlUnit)**: Theo dõi và điều chỉnh các tham số khác như nhiên liệu và hiệu suất dựa trên nhiệt độ và tốc độ động cơ.

3. **Chương trình chính (main)**:
   - Khởi tạo đối tượng Engine và các đối tượng quan sát (Observers) như CoolingSystem, WarningSystem, và EngineControlUnit.
   - Đăng ký các đối tượng quan sát vào đối tượng Engine.
   - Khi có thay đổi về nhiệt độ hoặc tốc độ, đối tượng Engine sẽ thông báo cho tất cả các hệ thống đã đăng ký để chúng có thể xử lý sự kiện tương ứng.

Mô tả chi tiết:

1. Thiết kế lớp Subject (Engine):

- **Nhiệm vụ**: Lớp này sẽ đóng vai trò là chủ thể theo dõi và thông báo cho các hệ thống khác về thay đổi trạng thái (ví dụ: nhiệt độ hoặc tốc độ động cơ).
- **Chức năng**:
  - **addObserver(Observer*)**: Thêm một đối tượng quan sát (Observer) vào danh sách theo dõi.
  - **removeObserver(Observer*)**: Gỡ bỏ một đối tượng quan sát khỏi danh sách theo dõi.
  - **notifyObservers()**: Thông báo cho tất cả các đối tượng quan sát về thay đổi trạng thái.
  - **setEngineState(int speed, float temperature)**: Thay đổi trạng thái của động cơ (tốc độ, nhiệt độ) và kích hoạt thông báo đến các hệ thống quan sát.
  - **getState()**: Trả về trạng thái hiện tại của động cơ (tốc độ và nhiệt độ).

2. Thiết kế các lớp Observer:

- **Nhiệm vụ**: Các hệ thống như CoolingSystem, WarningSystem, và EngineControlUnit sẽ đăng ký để nhận thông báo khi trạng thái của động cơ thay đổi. Mỗi hệ thống sẽ xử lý thông tin theo cách riêng của mình.
- **Chức năng chung**:
    - **update(int speed, float temperature)**: Phương thức được gọi bởi Engine khi có sự thay đổi về tốc độ hoặc nhiệt độ.
- **Cụ thể**:
    - **CoolingSystem**: Theo dõi nhiệt độ động cơ. Khi nhiệt độ vượt ngưỡng an toàn (ví dụ: 90 độ C), hệ thống sẽ bật quạt làm mát.
    - **WarningSystem**: Kích hoạt cảnh báo trên bảng điều khiển khi nhiệt độ vượt quá 100 độ C hoặc tốc độ vượt quá 6000 RPM.
    - **EngineControlUnit**: Điều chỉnh các tham số hiệu suất và nhiên liệu khi tốc độ hoặc nhiệt độ động cơ thay đổi.

3. Chương trình chính (main):

- **Bước 1**: Tạo đối tượng Engine (Subject) và các đối tượng quan sát (Observers) như CoolingSystem, WarningSystem, và EngineControlUnit.
- **Bước 2**: Đăng ký các hệ thống quan sát vào động cơ.
- **Bước 3**: Khi nhiệt độ hoặc tốc độ động cơ thay đổi, hệ thống sẽ kích hoạt thông báo đến các đối tượng quan sát. Mỗi đối tượng sẽ thực hiện hành động tương ứng dựa trên thông tin nhận được.

Yêu cầu nâng cao:

1. **Kiểm tra và kích hoạt sự kiện quá nhiệt**:
    - Nếu nhiệt độ động cơ vượt quá 100 độ C, hệ thống cảnh báo sẽ thông báo ngay lập tức cho người lái xe. Nếu nhiệt độ giảm xuống dưới mức này, cảnh báo sẽ được tắt.
2. **Mở rộng thêm các hệ thống quan sát khác**:
    - **FuelInjectionSystem**: Điều chỉnh lượng nhiên liệu được bơm vào động cơ dựa trên tốc độ hiện tại và nhiệt độ của động cơ.
    - **TransmissionControlUnit**: Điều chỉnh hộp số tự động dựa trên tốc độ của động cơ.
3. **Hủy đăng ký (Unsubscribe)**:
    - Khi một hệ thống không còn cần theo dõi động cơ, nó có thể hủy đăng ký khỏi danh sách quan sát để giảm tải cho hệ thống điều khiển trung tâm.
4. **Tối ưu hóa thông báo**:

- Khi có sự thay đổi nhỏ không ảnh hưởng đến trạng thái tổng thể của động cơ (ví dụ: thay đổi nhỏ trong khoảng nhiệt độ an toàn), hệ thống không cần thông báo cho các đối tượng quan sát để tránh quá tải và xử lý không cần thiết.